

**Session 2 Speaker Script — COMPLETE WITH FULL TALKING POINTS**

# LLM Fundamentals and Output Behavior

**Presenter:** Dinesh Wadhvani | **Session:** 2 of 12 | **Duration:** ~60 minutes

**Purpose:** Complete spoken script with full talking content to fill each slide's allocated time

| Slide | Title                      | Time    |
|-------|----------------------------|---------|
| 1     | Title Slide                | ~1 min  |
| 2     | Today's Journey            | ~2 min  |
| 3     | LLM Recap                  | ~5 min  |
| 4     | Tokens & Tokenization      | ~8 min  |
| 5     | Transformers Architecture  | ~8 min  |
| 6     | Pretraining vs Fine-tuning | ~8 min  |
| 7     | Context Windows            | ~8 min  |
| 8     | Parameter Settings         | ~8 min  |
| 9     | Lab Overview               | ~3 min  |
| 10    | Key Takeaways              | ~5 min  |
| 11    | Session 3 Preview          | ~2 min  |
| —     | Lab Exercise               | ~12 min |

## 01. TITLE SLIDE

~1 minute

### **WHAT'S ON SCREEN**

Session 2 label (blue) • Title: 'LLM Fundamentals and Output Behavior' • Session info: 'Session 2 of 12 | 60 minutes' • Dark blue background

### **SPEAKING SCRIPT**

Welcome back, everyone. I'm glad to see you here for Session 2. Last week we built your conceptual foundation—the history of AI, the evolution ladder from traditional AI all the way to agentic AI, and the vocabulary that lets you look at any AI product and categorize what it actually is.

Today is different. We're opening the hood. We're looking at how large language models actually work underneath all that polish. No heavy mathematics—I promise—just real, usable mental models. Models that explain why changing a single word in your prompt sometimes completely changes the output. Models that explain why temperature matters. Models that explain why these systems behave the way they do.

By the end of this hour, you won't just know what an LLM is. You'll understand why it behaves the way it does. That's the foundation for everything starting in Session 3 when we get into prompt engineering. Let's get started.

### **DELIVERY TIPS**

Keep energy high. • Pause after 'why changing a word changes output.' • Optional: Show of hands on who's used ChatGPT.

## 02. TODAY'S JOURNEY

~2 minutes

### **WHAT'S ON SCREEN**

8-item agenda with time badges • Items: Recap, Tokens, Transformers, Training, Context, Parameters, Lab, Wrap-up

### **SPEAKING SCRIPT**

Let me walk you through our hour so you always know where we are. We start with a five-minute recap of what an LLM is—just to reset everyone. Then eight minutes on tokens and tokenization—how text gets broken into pieces the model processes. This is where you'll understand why exact wording matters.

Eight more minutes on transformers—the architecture inside modern LLMs. We'll understand it conceptually. Then the training story: pretraining versus fine-tuning. Why a model can know general things but needs specialization for specific tasks.

Then context windows—what's the limit on input size? Why does this matter for cost and strategy? Then parameter settings like temperature—why some outputs are creative and others deterministic.

After all that theory, the lab. You'll take your Session 1 resume and run it three different ways with different prompt phrasings. Then experiment with temperature. You'll see firsthand how sensitive these systems are.

Then key takeaways and preview of Session 3. Notice the pattern: concept, mechanism, hands-on application. If we run long on discussion, I'll compress the recap before I ever touch lab time. That time is protected.

### **DELIVERY TIPS**

Move briskly. • Point to slides as you speak. • Emphasize 'protected lab time.'

## 03. QUICK RECAP: WHAT IS AN LLM?

~5 minutes

### **WHAT'S ON SCREEN**

Left: Definition + bullets • Right: 'Statistical pattern-matcher, not fact retriever' • Bottom: Next-token mechanism

### **SPEAKING SCRIPT**

Let's reset. An LLM—Large Language Model—is a deep learning model trained on enormous amounts of text. Billions of documents. Books, websites, code, academic papers, Reddit—basically the entire internet up to a certain date.

Its three goals are: understand patterns in language, predict the next most likely token given everything before, and generate coherent text by repeating that prediction.

Here's the critical insight: an LLM is fundamentally a statistical pattern-matcher, not a fact retriever. It's not pulling from a database. It's predicting what's statistically likely to come next based on training patterns. Usually correct—because correct information is the common pattern in huge text. But not always. That gap between 'statistically likely' and 'actually true' is why Retrieval-Augmented Generation exists. We'll cover that in Sessions 8 and 9.

The mechanism: your prompt comes in, tokenized into small pieces. The model processes tokens through its network, predicts the next most likely token, adds it to the sequence, then repeats. Token after token until response is complete.

Think autocomplete on your phone, but massively more sophisticated. Your phone learns from your texts. An LLM learns from billions of documents. That's the whole mechanism. Everything else we discuss today builds on this foundation.

### **DELIVERY TIPS**

Pause after 'pattern-matcher, not fact-retriever.' • Use autocomplete analogy. • Slow down when explaining mechanism—this is foundational.

## 04. TOKENS & TOKENIZATION

~8 minutes

### WHAT'S ON SCREEN

Token definition • Example: 'The model thinks.' → 4 tokens • Four colored boxes • Why It Matters: cost, context, sensitivity

### SPEAKING SCRIPT

A token is the smallest unit an LLM processes. Usually 3-4 characters on average, but that's just average. The reality is messier.

Example: 'The model thinks.' becomes four tokens: 'The', 'model', 'thinks', '.'. Makes sense. But tokenization is complex. Different models tokenize differently.

Same question, three ways: 'What is AI?' vs 'What is A.I.?' vs 'What is artificial intelligence?' Same meaning. Completely different tokenization. 'A.I.' might split into separate tokens. 'Artificial intelligence' is two words. Different patterns.

Sometimes different tokenizations lead to output differences. The model's attention layers focus on different parts depending on tokenization. Sometimes subtle, sometimes significant.

Why does this matter? Three reasons:

**Reason One: Cost.** Most APIs charge per token. If your use case tokenizes inefficiently, you might overpay 20-50% without realizing. You think '500 words' but it's actually 750 tokens.

**Reason Two: Context windows.** The model sees a certain number of tokens. Rare words, technical terms, non-English names tokenize inefficiently. You burn through your window faster.

**Reason Three:** Tokenization is so sensitive that tiny wording changes cause completely different tokenization patterns. That changes which parts the attention layers focus on. That cascades into different outputs. Not magic. Mechanistic.

Example: 'Write a creative poem' vs 'Write a poem, be creative' vs 'I need a creative poem. Topic: poetry.' Same request. Different tokenization. Different attention patterns. Sometimes subtly different outputs, sometimes dramatically different.

Real example from my work: 'machine learning algorithms' vs 'ML algorithms'. 'Machine learning' is two tokens, 'ML' is one. Different token sequences produce different recommendations. Not wildly different, but noticeably different emphasis. This is why prompt engineering is a real skill. You're not just being polite to the model. You're affecting how it processes your input mechanistically.

### DELIVERY TIPS

Emphasize weirdness. • Give real examples with punctuation. • Slow when explaining how tokenization affects output. • Flag lab connection.

## 05. TRANSFORMERS ARCHITECTURE (CONCEPTUALLY)

~8 minutes

### WHAT'S ON SCREEN

Flow: Input Tokens → Embedding → Attention Layers → Output Layer • 'The Attention Magic' section • Why rephrasing changes output

### SPEAKING SCRIPT

The architecture inside almost every modern LLM is called a transformer. Let me explain what it is and why it matters.

A transformer takes tokenized input and passes it through layers. Like an assembly line. Tokens go in one end, get processed through stages, probability distribution over next tokens comes out.

**Stage One: Input.** Prompt comes in as tokens. 'The model thinks.' becomes four tokens.

**Stage Two: Embedding.** Each token converts to a vector—a list of numbers. Think of it as translating tokens into rich 'representations' the neural network works with. Similar tokens get similar vectors. 'Cat' and 'kitten' have similar vectors—semantically related. 'Cat' and 'spaceship' have very different vectors.

**Stage Three: Attention Layers.** This is where magic happens. Transformers have multiple layers—often dozens—performing 'attention.' Attention looks at all tokens in your input and figures out which ones matter for predicting what comes next.

Key thing: this is NOT sequential. Not reading one token at a time. Older neural networks did that. Transformers are different. Attention lets the model look at ALL relationships at once. All in parallel. Which parts of my prompt are most relevant? Beginning? End? Specific phrase?

Analogy: Reading an article and someone asks a question. You don't read sequentially. Your attention goes immediately to relevant parts. Transformers do something similar with attention layers. Looking at all tokens in parallel, figuring out which ones matter.

**Stage Four: Output Layer.** After attention layers, model produces probability distribution over all possible next tokens. Essentially saying: given everything I've seen, here's how likely each token is. Then it picks the most likely or samples randomly—depends on your settings.

Why does this architecture matter? Because attention is why rephrasing prompts dramatically changes output. 'Write a creative poem' vs 'Write a poem, be creative'—different tokenization, different word order, attention layers focus on different parts. Downstream predictions shift. Outputs diverge.

Not a bug. Not randomness. Direct consequence of how attention and transformers work. Wording is mechanical. It affects how the model processes input at fundamental level. The textbook analogy: if someone highlights different sentences, you write a different summary even though content is the same. Because different parts got your attention. Same with transformers.

### DELIVERY TIPS

Spend real time on attention—this explains prompt sensitivity. • Use textbook/highlighting analogy. • Don't get into math. • Connect back to prompt sensitivity explicitly.

## 06. PRETRAINING VS FINE-TUNING

~8 minutes

### WHAT'S ON SCREEN

Left column: Pretraining • Right column: Fine-tuning • Bottom: Teaching analogy

### SPEAKING SCRIPT

Two distinct phases: pretraining and fine-tuning. Fundamentally different, different costs, different purposes. Understanding the difference explains what models can and can't do.

#### PRETRAINING: Expensive Phase

Big companies—OpenAI, Anthropic, Meta, Google—spend billions. They take massive text amounts. Internet. Books, websites, academic papers, code repositories, GitHub, Reddit—billions of documents. Petabytes of text.

Training on one task: predict the next token. That's it. Given token sequence, predict what comes next. Training for weeks on massive GPU clusters. Model has billions of parameters—adjustable weights. Learning through next-token prediction on billions of documents.

What emerges: 'foundation model.' General-purpose. Learned broad patterns in language. Knows English grammar, math, coding, science, history, literature—everything common in training data.

But—important—relatively unfiltered. Might output toxic content. Might hallucinate facts. Might not follow instructions well. Smart but not necessarily helpful or safe.

#### FINE-TUNING: Specialization Phase

Fine-tuning takes pretrained model—that foundation with general knowledge—and does second, much shorter training on smaller, curated dataset. What data? Depends on purpose.

For helpful assistant like ChatGPT: labeled examples of good Q&A, examples showing helpfulness, feedback from human raters: 'This response was good. This one was bad. This one was offensive.'

Model learns. Parameters adjusted. Pretrained knowledge stays. Behavior gets shaped toward specific purpose.

Fine-tuning is vastly cheaper and faster than pretraining. Not starting from scratch. Starting from model that already understands language. Just pointing it in specific direction.

#### Why This Matters

When ChatGPT writes code well—GitHub Copilot suggests entire functions—that's because model was pretrained on massive code amounts. Billions of lines. Learned programming patterns during pretraining.

Why might same model struggle with specialized legal terminology? Probably not fine-tuned on legal examples. Or legal domain wasn't heavily represented in pretraining data.

Concrete example: company wanted specialized medical AI. Took foundation model, fine-tuned on medical literature, medical case studies, curated medical Q&A. Suddenly much better at medical reasoning. Not different model. Not retrained from scratch. Just specialized.

#### Teaching Analogy

Pretraining is like school. Learn general knowledge. Math, history, science, language. Broad foundation. Fine-tuning is like taking masterclass in your specialty. Already know basics. Now deepening expertise in specific area.

### DELIVERY TIPS

Use teaching analogy. • Emphasize scale for pretraining, contrast for fine-tuning. • Give concrete examples. • Preview RAG for future sessions.

## 07. CONTEXT WINDOWS

~8 minutes

### WHAT'S ON SCREEN

Definition accent box • Real examples table • Why It Matters bullets

### SPEAKING SCRIPT

Context window is the maximum number of tokens the model can read in a single request. Includes your prompt AND model's response.

Real examples matter: GPT-3.5 has 4,096 tokens—roughly 3,000 words. GPT-4 has 8,192 tokens—roughly 6,000 words. Claude has 200,000 tokens—roughly 150,000 words. Massive difference. This isn't small detail. Fundamental constraint shaping how you use these models.

### Why Does This Matter? Three Reasons:

#### Reason One: Large Documents Don't Fit

Lawyer with 50-page contract. Want to ask: 'What are key risks?' With 8,192 token window, you can't fit 50-page contract in one request. Contract is probably 100,000 tokens. Way over limit.

What do you do? Chunk it. Break into pieces. Send first 20 pages, ask risks. Next 20 pages. Stitch together insights. Doable but complex. Requires strategy.

With Claude's 200,000 tokens, entire contract might fit. One question. Comprehensive answer. Different workflows enabled by different context windows.

#### Reason Two: Cost Implications

Most LLM APIs charge per token. Not per request. Per token. Pay OpenAI \$0.01 per thousand tokens. Send document three times (chunks) vs once? Paying three times as much.

That's why smart retrieval strategies matter. Instead of sending entire document, send only relevant excerpt. Use search or semantic matching to identify relevant parts, send just those plus question.

Fit everything in context window. Pay less. Faster responses.

#### Reason Three: Driving Innovation

Limited context window problem spawned whole class of solutions. Most important: Retrieval-Augmented Generation, or RAG. We'll dive deep in Sessions 8 and 9. Short version:

RAG is technique where you retrieve relevant information from database, stuff that into prompt alongside question, ask model to answer based on context. Ground output in real, verified information. Not relying on training. Giving actual documents to reference.

RAG exists specifically because of context window constraints. If windows were infinite, just send entire database every time. They're not. So retrieve intelligently.

#### Future: Windows Growing but Still Constrained

Context windows expanding rapidly. Year ago, 4,000 tokens was standard. Now companies pushing 128,000, 200,000, larger. Why? Longer windows enable more capabilities. Longer conversations without losing context. Analyze entire codebases. Feed entire books.

But they get expensive. Processing 200,000 token request costs way more than 4,000 token request. Always tradeoff. Key insight: context windows are fundamental constraint. They shape what's possible. They shape workflows you design. They shape techniques we use—like RAG—to work around limitations.

***DELIVERY TIPS***

Give concrete numbers and examples. • Lawyer/contract example is relatable. • Emphasize three implications. • Preview RAG.

## 08. PARAMETER SETTINGS: CONTROLLING OUTPUT

~8 minutes

### WHAT'S ON SCREEN

Temperature spectrum • Top-p explanation • Max tokens • Practical examples

### SPEAKING SCRIPT

When you call an LLM, you send settings that control how it generates. Hyperparameters or sampling parameters. Levers you turn to control behavior.

#### TEMPERATURE: Controlling Randomness

Temperature ranges 0.0 to 2.0 (usually capped at 1.0). Controls randomness in predictions. At each step, model has probability distribution over possible next tokens. Temperature controls how 'sharp' or 'flat' that distribution is.

##### At 0.0: Deterministic

Always picks single most likely token. Every time. Run same prompt twice, get identical outputs. Reproducible. Consistent. Use for reliability: customer service, factual Q&A, information retrieval. Same answer every time.

##### At 1.0: Natural

Samples naturally from learned probabilities. No artificial manipulation. Often considered 'default' or 'natural' temperature.

##### At High—1.2, 1.5, 2.0: Creative

Model gets random. Unlikely tokens become more likely. Distribution flattens. More variety, more creativity, more nonsense. Use for creative writing, brainstorming, generating multiple ideas. Trading accuracy for creativity.

Concrete example: Ask both extremes 'Write short story about robot.' At 0.3, get conventional, predictable story. Probably sci-fi elements, familiar narrative patterns. What model calculated most likely. At 1.5, might get weird story. Unexpected plot twists, unusual motivations, surprising endings. Unlikely tokens selected more often.

#### TOP-P: Different Diversity Control

Top-p (nucleus sampling) ranges 0.0 to 1.0. Controls diversity differently than temperature. Instead of controlling randomness directly, says: 'Only consider tokens accounting for top p% of probability.'

At 0.9: only tokens making up 90% of probability. Ignore 10% tail. At 0.5: only 50%. Very restrictive. At 1.0: all tokens. Different combinations give different effects. Temperature controls 'how random,' top-p controls 'which tokens to consider.'

#### MAX TOKENS: Hard Length Limit

Straightforward: hard ceiling on tokens model generates before stopping. Set max\_tokens=500, stops at 500 tokens. Even mid-sentence sometimes. Use to: prevent runaway generations, control costs, force conciseness.

#### Practical Takeaways

These parameters directly affect what you get. Same prompt. Temperature 0.3 vs 1.5? Completely different responses. Not bug. Feature. Tune based on needs: factual output—low temperature; creative output—high temperature; concise—low max\_tokens; varied—high top-p.

Lab connection: you're about to run resume prompt with temperature 0.3 and 1.2. Literally see how much difference temperature makes. You'll be shocked. Visceral understanding, not just intellectual.

#### DELIVERY TIPS

Use concrete numbers. • Point to spectrum. • Build anticipation for lab. • These are not mysterious.

## 09. LAB EXERCISE OVERVIEW

~3 minutes

### **WHAT'S ON SCREEN**

Lab Exercise title • 4 numbered steps • Deliverable accent box

### **SPEAKING SCRIPT**

Moving into hands-on. Everything we just talked about becomes real. You'll see it in action. Twelve minutes, everyone actively working.

**Step One:** Find your Session 1 resume and fictional profile. Created them last week? Great. Don't have them? Spend 30 seconds recreating profile. Name, degree, skills, one internship.

**Step Two:** Generate three prompt variations. Don't run same prompt three times. Actually rephrase.

Variation One—detailed formal: 'Write comprehensive professional resume. Include achievements. Clear sections.'

Variation Two—conversational casual: 'Resume coach here helping student. Let me write resume showing personality while staying professional.'

Variation Three—structured: 'Generate resume using exact structure: [PROFILE] [SKILLS] [EXPERIENCE] [EDUCATION] [PROJECTS]. Keep bullets short.'

**Step Three:** Compare outputs. What changed? Content? Structure? Tone? Length? One produce better formatting? One more detailed? Different emphases?

**Step Four:** Temperature experiment. Pick one prompt. Run twice. Low temperature—0.3. High temperature—1.2+. Compare. Did facts stay same? Did wording change? Which would you actually use?

**Deliverable:** Document with three prompt variations, three outputs (or summaries), observations on differences, temperature comparison.

Why? This lab builds intuition. You'll see LLM output sensitivity to wording. See temperature effects. Understand why prompt engineering is real skill. In Session 3 when we discuss engineering, you'll anchor on hands-on experience here.

Go. Twelve minutes. I'll circulate. Flag me if tool isn't working, you need temperature help, or want sounding board.

### **DELIVERY TIPS**

Clear and concise. • Numbered steps. • Emphasize intuition building. • Let them know you'll circulate.

## 10. KEY TAKEAWAYS FROM SESSION 2

~5 minutes

### **WHAT'S ON SCREEN**

7 numbered takeaway statements • Alternating backgrounds • Each quotable and standalone

### **SPEAKING SCRIPT**

#### **Takeaway One: LLMs Predict Tokens**

LLMs predict next token. Not retrieving facts from database. Not looking things up. Generating statistically likely text. This fundamental fact explains almost every LLM quirk, including why they can be confidently wrong. Remember: statistical likelihood is different from truth.

#### **Takeaway Two: Tokens are Currency**

Tokens drive cost—you pay per token. Drive context limits—windows measured in tokens. Drive output sensitivity. Different tokenizations lead to different attention patterns and different outputs. Wording matters because tokenization matters.

#### **Takeaway Three: Transformers Use Attention**

Transformers use attention to understand relationships between tokens in prompt. Attention is parallelized—not sequential. Revolutionary thing about transformers—they look at all relationships at once. Tiny wording changes shift which parts attention layers focus on. Cascades into different responses.

#### **Takeaway Four: Pretraining vs Fine-Tuning**

Pretraining builds general knowledge. Fine-tuning specializes it. Pretraining happens once on billions of documents. Fine-tuning happens repeatedly on specialized datasets. Why ChatGPT is good at customer service (fine-tuned) and GPT-3 base isn't (not fine-tuned).

#### **Takeaway Five: Context Windows Constrain You**

Context windows limit how much text fits in one request. Range from 4,000 to 200,000 tokens. Affects workflow, cost, what's possible. Limited window problem drives innovation—RAG is direct solution.

#### **Takeaway Six: Parameters Control Behavior**

Parameters like temperature directly control output. Low temperature means focused, deterministic. High temperature means creative, varied. These aren't mysterious. Simple levers you turn.

#### **Takeaway Seven: Wording Fundamentally Changes Output**

Tiny wording changes shift which tokens model attends to. Rephrasing isn't stylistic—it's mechanical. Fundamentally changes response. You've seen this in lab. You'll use this in every future session—prompt engineering, structured outputs, retrieval systems, agents.

### **DELIVERY TIPS**

Pause after each takeaway. • These are quotable—refer back in future sessions. • Optional: Ask for examples or show of hands.

## 11. COMING UP NEXT

~2 minutes

### **WHAT'S ON SCREEN**

Dark blue background • Title: Session 3: Prompt Engineering Fundamentals • Preview bullets • Closing

### **SPEAKING SCRIPT**

Session 3 is where theory becomes practice. We established conceptually what LLMs are. We opened the hood—tokens, transformers, attention, pretraining, fine-tuning, context windows, parameters.

Session 3: we learn how to write prompts that work. There are actual patterns. Structures that reliably produce better outputs. Not magic. Technique.

We'll explore few-shot learning—why showing model examples helps it understand better than just explaining. We'll talk about common pitfalls and why certain phrasings backfire. We'll reverse-engineer what works and what doesn't.

Lab: you'll take that Session 1 resume—the one you just experimented with today—and craft it into polished prompt. One that reliably produces high-quality output.

Everything you learned today about tokens, attention, parameters? You'll put to work next session. You'll understand not just what works but why.

Thank you all for engaging today. Questions were great. Energy was great. Genuinely enjoyed this. See you next time.

### **DELIVERY TIPS**

Keep brief and warm. • Build momentum for Session 3. • End with appreciation and energy.